

Tvorba mobilních aplikací pro Android

7. seminář

Matěj Dostál

Univerzita Palackého v Olomouci

25. 3. 2026

Mobilní aplikace UX

- Typická Android aplikace sestává z několika komponent (*activity, fragmenty, služby, content nebo broadcast providers*)
- ⇒ Uživatel může interagovat s více aplikacemi v krátkém čase – aplikace se musí umět přizpůsobit různým user-driven workflows.
- Musíme myslet také na to, že telefony mají omezené zdroje, takže OS může aplikaci kdykoli zabít, aby uvolnil místo

Mobilní aplikace UX

- Typická Android aplikace sestává z několika komponent (*activity, fragmenty, služby, content nebo broadcast providers*)
- ⇒ Uživatel může interagovat s více aplikacemi v krátkém čase – aplikace se musí umět přizpůsobit různým user-driven workflows.
- Musíme myslet také na to, že telefony mají omezené zdroje, takže OS může aplikaci kdykoli zabít, aby uvolnil místo
- Je tedy možné, že komponenty naší aplikace mohou být spuštěny individuálně a „out-of-order“, stejně jako že je OS kdykoli zničí

Mobilní aplikace UX

- Typická Android aplikace sestává z několika komponent (*activity, fragmenty, služby, content nebo broadcast providers*)
- ⇒ Uživatel může interagovat s více aplikacemi v krátkém čase – aplikace se musí umět přizpůsobit různým user-driven workflows.
- Musíme myslet také na to, že telefony mají omezené zdroje, takže OS může aplikaci kdykoli zabít, aby uvolnil místo
- Je tedy možné, že komponenty naší aplikace mohou být spuštěny individuálně a „out-of-order“, stejně jako že je OS kdykoli zničí
- Toto nemůžeme nijak ovlivnit ani kontrolovat – musíme se tomu přizpůsobit a neuchovávat data nebo stav v komponentách aplikace a ty by na sobě neměly záviset

Architektura aplikací

- S rostoucí velikostí aplikace je potřeba definovat architekturu, která umožní aplikaci dále rozšiřovat, udělá ji robustnější a zjednoduší testování
- Architektura definuje hranice mezi částmi aplikace a zodpovědnost, kterou by každá část měla mít

Architektura aplikací

- S rostoucí velikostí aplikace je potřeba definovat architekturu, která umožní aplikaci dále rozšiřovat, udělá ji robustnější a zjednoduší testování
- Architektura definuje hranice mezi částmi aplikace a zodpovědnost, kterou by každá část měla mít
- Abychom dosáhli výše zmíněného efektu, měli bychom nadesignovat naši aplikaci tak, aby následovala určité principy
- Není třeba „znovu vynalézat kolo“

Architektura aplikací

- S rostoucí velikostí aplikace je potřeba definovat architekturu, která umožní aplikaci dále rozšiřovat, udělá ji robustnější a zjednoduší testování
- Architektura definuje hranice mezi částmi aplikace a zodpovědnost, kterou by každá část měla mít
- Abychom dosáhli výše zmíněného efektu, měli bychom nadesignovat naši aplikaci tak, aby následovala určité principy
- Není třeba „znovu vynalézat kolo“
- Kromě návrhových vzorů (Singleton, Adapter, Factory, Builder, ...) máme osvědčené vzory pro architekturu aplikací
- Vhodnost architektury se liší podle typu aplikace

Oddělení starostí

- Běžnou chybou dříve bylo psát všechnen kód do Aktivit či Fragmentů – tyto UI třídy by měly obsahovat pouze zobrazení a interakci s OS
- V moderním pojetí (Jetpack Compose) píšeme UI pomocí `@Composable` funkcí, ale princip zůstává: logika do UI nepatří
- Navíc musíme myslet na to, že UI komponenty „nevlastníme“ – slouží jen jako spojení mezi OS a naší aplikací $\Rightarrow$ mohou být zničeny $\Rightarrow$ minimalizace závislosti na nich
- Díky tomuto rozdělení můžeme mnohem lépe testovat kód

Oddělení starostí

- Běžnou chybou dříve bylo psát všechnen kód do Aktivitu či Fragmentů – tyto UI třídy by měly obsahovat pouze zobrazení a interakci s OS
- V moderním pojetí (Jetpack Compose) píšeme UI pomocí `@Composable` funkcí, ale princip zůstává: logika do UI nepatří
- Navíc musíme myslet na to, že UI komponenty „nevlastníme“ – slouží jen jako spojení mezi OS a naší aplikací $\Rightarrow$ mohou být zničeny $\Rightarrow$ minimalizace závislosti na nich
- Díky tomuto rozdělení můžeme mnohem lépe testovat kód

- Na platformě Android je velmi doporučená architektura *Model-View-ViewModel* (MVVM), která nahradila starší MVC nebo MVP
 - ▶ Mírné odlišnosti v chápání rolí
 - ▶ Jasné rozdělení zodpovědnosti za data a za zobrazení

Model-View-Controller (MVC) – Historický koncept

- Model – Aplikační data a veškerá logika (nezávislá na zobrazení)
 - ▶ Databázové entity
 - ▶ Interní stav aplikace
 - ▶ Veškerá funkcionality

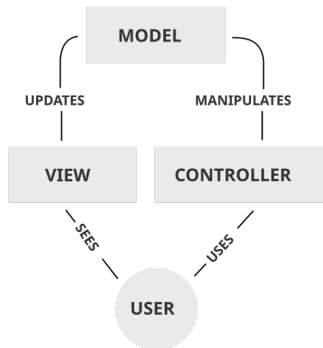
Model-View-Controller (MVC) – Historický koncept

- Model – Aplikační data a veškerá logika (nezávislá na zobrazení)
 - ▶ Databázové entity
 - ▶ Interní stav aplikace
 - ▶ Veškerá funkcionality
- View – Komponenty, které zobrazují data (která obdrží)
 - ▶ Logika pouze za účelem prezentace
 - ▶ „Hloupé zobrazovače“
 - ▶ Zasílání uživatelských akcí (klik, psaní, ...) kontroleru

Model-View-Controller (MVC) – Historický koncept

- Model – Aplikační data a veškerá logika (nezávislá na zobrazení)
 - ▶ Databázové entity
 - ▶ Interní stav aplikace
 - ▶ Veškerá funkcionality
- View – Komponenty, které zobrazují data (která obdrží)
 - ▶ Logika pouze za účelem prezentace
 - ▶ „Hloupé zobrazovače“
 - ▶ Zasílání uživatelských akcí (klik, psaní, ...) kontroleru
- Controller – Vrstva mezi Modelem a View
 - ▶ Přijímá uživatelské vstupy (validace) a předává je Modelu
 - ▶ Controller zná své View a přímo ho ovládá

MVC obrázkem



Obrázek: Zdroj: <https://en.wikipedia.org/wiki/File:MVC-Process.svg>

Model-View-ViewModel (MVVM)

- Model – Aplikační data a veškerá logika (zcela nezávislá na UI)
 - ▶ Repozitáře, databázové entity
 - ▶ Výpočty a byznys logika

Model-View-ViewModel (MVVM)

- Model – Aplikační data a veškerá logika (zcela nezávislá na UI)
 - ▶ Repozitáře, databázové entity
 - ▶ Výpočty a byznys logika
- View – Pouze zobrazovací komponenty (XML layouts nebo Compose funkce)
 - ▶ Žádná aplikační logika
 - ▶ Reaguje automaticky na změny stavu z ViewModelu

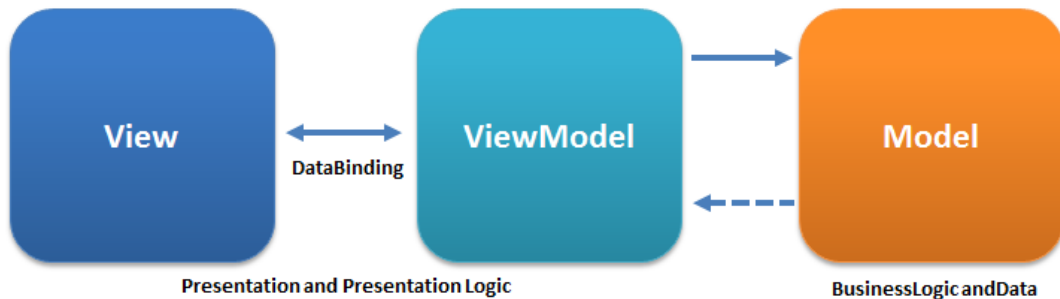
Model-View-ViewModel (MVVM)

- Model – Aplikační data a veškerá logika (zcela nezávislá na UI)
 - ▶ Repozitáře, databázové entity
 - ▶ Výpočty a byznys logika
- View – Pouze zobrazovací komponenty (XML layouty nebo Compose funkce)
 - ▶ Žádná aplikační logika
 - ▶ Reaguje automaticky na změny stavu z ViewModelu
- ViewModel – Drží a připravuje data, která se mají zobrazit
 - ▶ **Nezná své View** – pouze vystavuje reaktivní data (např. StateFlow)
 - ▶ „Překlápí“ uživatelské akce (přijaté z View) do Modelu
 - ▶ Přechází otočení obrazovky a změny konfigurace

Model-View-ViewModel (MVVM)

- Model – Aplikační data a veškerá logika (zcela nezávislá na UI)
 - ▶ Repozitáře, databázové entity
 - ▶ Výpočty a byznys logika
- View – Pouze zobrazovací komponenty (XML layouts nebo Compose funkce)
 - ▶ Žádná aplikační logika
 - ▶ Reaguje automaticky na změny stavu z ViewModelu
- ViewModel – Drží a připravuje data, která se mají zobrazit
 - ▶ **Nezná své View** – pouze vystavuje reaktivní data (např. StateFlow)
 - ▶ „Překlápí“ uživatelské akce (přijaté z View) do Modelu
 - ▶ Přechází otočení obrazovky a změny konfigurace
- Rozdělení rolí ⇒ lepší testovatelnost
 - ▶ Můžeme ViewModel a logiku testovat bez nutnosti spouštět Android emulátor

MVVM obrázkem



Obrázek: Zdroj: <https://en.wikipedia.org/wiki/File:MVVMPattern.png>

MVVM v Androidu – Evoluce

- Ve starém (imperativním) vývoji jsme museli prvky hledat (např. přes `findViewById`) a ručně jim měnit hodnoty

MVVM v Androidu – Evoluce

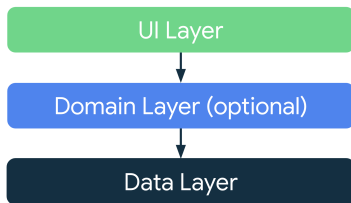
- Ve starém (imperativním) vývoji jsme museli prvky hledat (např. přes `findViewById`) a ručně jim měnit hodnoty
- Dnes (v Jetpack Compose) je uživatelské rozhraní plně řízeno **stavem (State)**, který je oddělený od UI
 - ▶ Nedojde ke ztrátě dat, když OS „zabije“ aplikaci
 - ▶ UI se samo překreslí při jakékoliv změně dat ve ViewModelu

MVVM v Androidu – Evoluce

- Ve starém (imperativním) vývoji jsme museli prvky hledat (např. přes `findViewById`) a ručně jim měnit hodnoty
- Dnes (v Jetpack Compose) je uživatelské rozhraní plně řízeno **stavem (State)**, který je oddělený od UI
 - ▶ Nedojde ke ztrátě dat, když OS „zabije“ aplikaci
 - ▶ UI se samo překreslí při jakékoliv změně dat ve ViewModelu
- Je doporučováno rozdělit aplikaci na 2 vrstvy – UI vrstva a Datová vrstva (plus volitelně Doménová vrstva)

MVVM v Androidu – Evoluce

- Ve starém (imperativním) vývoji jsme museli prvky hledat (např. přes `findViewById`) a ručně jim měnit hodnoty
- Dnes (v Jetpack Compose) je uživatelské rozhraní plně řízeno **stavem (State)**, který je oddělený od UI
 - ▶ Nedojde ke ztrátě dat, když OS „zabije“ aplikaci
 - ▶ UI se samo překreslí při jakékoliv změně dat ve ViewModelu
- Je doporučováno rozdělit aplikaci na 2 vrstvy – UI vrstva a Datová vrstva (plus volitelně Doménová vrstva)



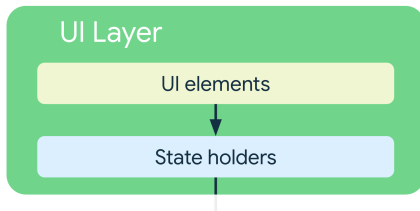
Obrázek: Zdroj: <https://developer.android.com/jetpack/guide>

Vrstva UI – prezentační vrstva

- Zobrazuje aplikační data na displeji
- Při změně dat, ať už interakcí s uživatelem nebo jiným externím inputem (např. odpověď serveru), se UI automaticky aktualizuje

Vrstva UI – prezentační vrstva

- Zobrazuje aplikační data na displeji
- Při změně dat, ať už interakcí s uživatelem nebo jiným externím inputem (např. odpověď serveru), se UI automaticky aktualizuje
- Sestává ze dvou hlavních částí:
 - ▶ UI elementy (v moderním pojetí `@Composable` funkce), které vykreslí obrazovku
 - ▶ State holder (třída `ViewModel`), který obsahuje data, drží stav a zpracovává logiku z UI

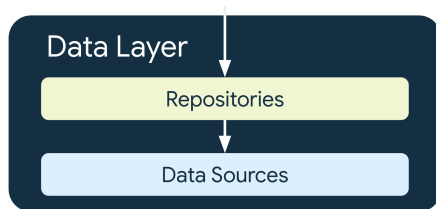


Datová vrstva

- Sdružuje aplikační data a veškerou aplikační (business) logiku
- Řeší, kam budou data uložena (lokální DB, síťový request) a jak budou transformována

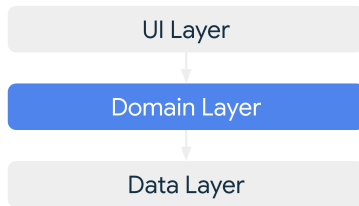
Datová vrstva

- Sdružuje aplikační data a veškerou aplikační (business) logiku
- Řeší, kam budou data uložena (lokální DB, síťový request) a jak budou transformována
- Je složena z *repozitářů* (typicky pro každou logickou entitu zvlášť)
 - ▶ Obsahují datové zdroje (nula až několik, např. Room DAO nebo Retrofit API)
 - ▶ Sdružují funkcionalitu okolo jednoho typu dat (např. `MoviesRepository` nebo `PaymentRepository`)
 - ▶ Repožitáře jsou zodpovědné za: poskytování dat zbytku aplikace, centralizované úpravy dat a abstrakci zdrojů



Doménová vrstva

- Nepovinná vrstva umístěná mezi UI a datovou vrstvou
- Zodpovědná za zapouzdření komplexní logiky, která se sdílí mezi více ViewModely (tzv. Use Cases / Interactors)
- Volitelná – velmi jednoduché aplikace ji nepotřebují, ale u komplexních projektů je standardem
- Výrazně odbourává duplicitu v kódu



Třída ViewModel

- Systém může zničit UI prvky (aktivity, fragmenty) při otočení displeje nebo nedostatku paměti
 - ▶ Bez správné architektury bychom ztratili stav rozepsaného formuláře atd.

Třída ViewModel

- Systém může zničit UI prvky (aktivity, fragmenty) při otočení displeje nebo nedostatku paměti
 - ▶ Bez správné architektury bychom ztratili stav rozepsaného formuláře atd.
- Časově náročnější operace navíc nesmíme volat z hlavního UI vlákna

Třída ViewModel

- Systém může zničit UI prvky (aktivity, fragmenty) při otočení displeje nebo nedostatku paměti
 - ▶ Bez správné architektury bychom ztratili stav rozepsaného formuláře atd.
- Časově náročnější operace navíc nesmíme volat z hlavního UI vlákna
- ⇒ Třída `ViewModel` (součást Android Jetpack)
- Je to komponenta, která přežívá běžný životní cyklus aktivit a překreslení Compose stromu
 - ▶ Ideální místo pro uložení stavu UI, který chceme zachovat
 - ▶ Nativní podpora asynchronních *Kotlin Coroutines* (přes `viewModelScope`)

Třída ViewModel

- Systém může zničit UI prvky (aktivity, fragmenty) při otočení displeje nebo nedostatku paměti
 - ▶ Bez správné architektury bychom ztratili stav rozepsaného formuláře atd.
- Časově náročnější operace navíc nesmíme volat z hlavního UI vlákna
- ⇒ Třída `ViewModel` (součást Android Jetpack)
- Je to komponenta, která přežívá běžný životní cyklus aktivit a překreslení Compose stromu
 - ▶ Ideální místo pro uložení stavu UI, který chceme zachovat
 - ▶ Nativní podpora asynchronních *Kotlin Coroutines* (přes `viewModelScope`)
- ViewModel je pevně vázán na životní cyklus svého *majitele* (Aktivita, Navigační graf) a zaniká až při jeho definitivním ukončení

Třída pro aplikaci

- Často potřebujeme inicializovat věci (jako lokální databázi) tak, aby byly dostupné napříč celou aplikací
- Podědíme proto z `android.app.Application` – třída pro správu globálního stavu aplikace

```
class MyApplication : Application() {  
    val database by lazy { AppDb.getDatabase(this) }  
}
```

- Do `AndroidManifest.xml` přidáme do elementu `application` odkaz na naši třídu

```
android:name=".MyApplication"
```

Reaktivní data – Historie (LiveData) vs Současnost (Flow)

- Pokud přidáme položku do DB, chceme, aby se seznam na displeji automaticky obnovil

Reaktivní data – Historie (LiveData) vs Současnost (Flow)

- Pokud přidáme položku do DB, chceme, aby se seznam na displeji automaticky obnovil
- Ve starším vývoji pro XML layouty se k tomu používala třída `LiveData`
 - ▶ Byla to Android-specifická třída chránící data a reagující na životní cyklus `View`

Reaktivní data – Historie (LiveData) vs Současnost (Flow)

- Pokud přidáme položku do DB, chceme, aby se seznam na displeji automaticky obnovil
- Ve starším vývoji pro XML layouts se k tomu používala třída `LiveData`
 - ▶ Byla to Android-specifická třída chránící data a reagující na životní cyklus View
- V moderním vývoji s Jetpack Compose (a čistým Kotlinem) se využívá **StateFlow** a **Flow**
 - ▶ Jedná se o nativní součást jazyka Kotlin (nepotřebuje Android knihovny pro byznys logiku)
 - ▶ V Jetpack Compose stav jednoduše „sbíráme“ a UI se samo překreslí, kdykoliv proud dat (Flow) vyemituje novou hodnotu

Příprava ViewModelu a Flow v Compose

- Zde je ukázka moderního ViewModelu, který využívá Kotlin Coroutines a Flow
- Z databáze získáme data jako `Flow` a převedeme je na `StateFlow`, které je ideální pro Compose

```
class MainViewModel(private val dao: CapybaraDao) : ViewModel() {
```

```
    // Udrzuje reaktivni stav, který Compose umi cist
    val capybaras: StateFlow<List<CapybaraItem>> = dao.getAll()
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.Lazily,
            initialValue = emptyList()
        )
}
```

Historické okénko – Data Binding (XML)

- Dříve, v době XML layoutů, se používala knihovna *Data Binding Library*
- Umožňovala vložit proměnné z ViewModelu přímo do XML kódu a zbavit se hledání prvků v Javě/Kotlinu:

```
<TextView  
    android:text="@{viewModel.userName}" />
```

Historické okénko – Data Binding (XML)

- Dříve, v době XML layoutů, se používala knihovna *Data Binding Library*
- Umožňovala vložit proměnné z ViewModelu přímo do XML kódu a zbavit se hledání prvků v Javě/Kotlinu:

```
<TextView
```

```
    android:text="@{viewModel.userName}" />
```

- Pro formuláře (obousměrný binding) se přidalo rovnítko @=:

```
<EditText
```

```
    android:text="@={viewModel.copyAddName}" />
```

Historické okénko – Data Binding (XML)

- Dříve, v době XML layoutů, se používala knihovna *Data Binding Library*
- Umožňovala vložit proměnné z ViewModelu přímo do XML kódu a zbavit se hledání prvků v Javě/Kotlinu:

```
<TextView  
    android:text="@{viewModel.userName}" />
```

- Pro formuláře (obousměrný binding) se přidalo rovnítko @=:

```
<EditText  
    android:text="@={viewModel.copyAddName}" />
```

- Tento přístup fungoval, ale vyžadoval těžkopádné generování kódu na pozadí, špatně se debugoval a občas vedl k nečitelným chybám při kompilaci

Moderní přístup – Jetpack Compose a State

- Jetpack Compose zcela nahrazuje XML a Data Binding elegantnějším přístupem v čistém Kotlinu
- Z ViewModelu vytvoříme jednosměrný tok dat (Unidirectional Data Flow)

Moderní přístup – Jetpack Compose a State

- Jetpack Compose zcela nahrazuje XML a Data Binding elegantnějším přístupem v čistém Kotlinu
- Z ViewModelu vytvoříme jednosměrný tok dat (Unidirectional Data Flow)
- Stav ve ViewModelu:

```
class FormViewModel : ViewModel() {  
    private val _copyName = MutableStateFlow("")  
    val copyName: StateFlow<String> = _copyName.asStateFlow()  
  
    fun updateName(newName: String) {  
        _copyName.value = newName  
    }  
}
```

- Data tečou z ViewModelu do UI, události tečou z UI do ViewModelu

Obousměrné propojení v Compose

- V Compose neexistuje skrytá "magie" pro binding; vše předáváme explicitně
- UI funkci stačí připojit k ViewModelu a přečíst stav pomocí `collectAsState()`

@Composable

```
fun CapybaraForm(viewModel: FormViewModel) {  
    // Kdykoliv se stav změní, toto UI se znovu překreslí  
    val name by viewModel.capyName.collectAsState()  
  
    TextField(  
        value = name,  
        onChange = { newValue ->  
            viewModel.updateName(newValue) // Odeslání akce  
        }  
    )  
}
```

- Tento přístup je bezpečnější, předvídatelnější a mnohem lépe se testuje

Dokončení MVVM – Zapojení Repozitáře

- V aplikaci bychom stále neměli přímo míchat operace pro DB s ViewModelem
- ViewModel by se měl starat pouze o stav pro UI, neměl by řešit nízkoúrovňové ukládání

Dokončení MVVM – Zapojení Repozitáře

- V aplikaci bychom stále neměli přímo míchat operace pro DB s ViewModelem
- ViewModel by se měl starat pouze o stav pro UI, neměl by řešit nízkoúrovňové ukládání
- Vytvoříme proto `CapybaraRepository`
 - ▶ Bude obsahovat veškerou aplikační logiku spojenou se získáváním nebo ukládáním dat
 - ▶ ViewModel bude už jen volat asynchronní metody tohoto repozitáře

```
class CapybaraRepository(private val dao: CapybaraDao) {  
    suspend fun createCapybara(name: String, imageld: Int) {  
        val capy = Capybaraltem(name = name, imageld = imageld)  
        dao.insert(capy)  
    }  
}
```

Dokončení MVVM – Zapojení Repozitáře

- V aplikaci bychom stále neměli přímo míchat operace pro DB s ViewModelem
- ViewModel by se měl starat pouze o stav pro UI, neměl by řešit nízkoúrovňové ukládání
- Vytvoříme proto `CapybaraRepository`
 - ▶ Bude obsahovat veškerou aplikační logiku spojenou se získáváním nebo ukládáním dat
 - ▶ ViewModel bude už jen volat asynchronní metody tohoto repozitáře

```
class CapybaraRepository(private val dao: CapybaraDao) {  
    suspend fun createCapybara(name: String, imageld: Int) {  
        val capy = Capybaraltem(name = name, imageld = imageld)  
        dao.insert(capy)  
    }  
}
```

- Tím získáme čistou MVVM architekturu. Zodpovědnost komponent je jasně oddělena a vše je snadno testovatelné

Organizace kódu aplikace

- S rostoucí velikostí aplikace je důležité správně organizovat soubory do složek (balíčků)
- Špatná struktura vede k nepřehlednému kódu a ztěžuje přidávání nových funkcí nebo opravy chyb

Organizace kódu aplikace

- S rostoucí velikostí aplikace je důležité správně organizovat soubory do složek (balíčků)
- Špatná struktura vede k nepřehlednému kódu a ztěžuje přidávání nových funkcí nebo opravy chyb
- V historii Android vývoje se uplatnily dva hlavní přístupy:
 - 1 **Package by Layer** (Sdružování podle vrstev) – historický a dnes nedoporučovaný přístup
 - 2 **Package by Feature** (Sdružování podle funkcionality) – moderní průmyslový standard

Detailní pohled na Package by Layer

- Tento přístup byl absolutním standardem v raných fázích Android vývoje
- Aplikace je zde rozdělena na několik hlavních balíčků čistě podle technického typu (vrstvy)

Detailní pohled na Package by Layer

- Tento přístup byl absolutním standardem v raných fázích Android vývoje
- Aplikace je zde rozdělena na několik hlavních balíčků čistě podle technického typu (vrstvy)
- Všechny ViewModely žijí v jedné složce, všechna DAO rozhraní ve druhé, všechny UI obrazovky ve třetí
- Z pohledu architektury to odpovídá našemu rozdělení (UI vrstva, Datová vrstva), takže to na první pohled dává smysl

Detailní pohled na Package by Layer

- Tento přístup byl absolutním standardem v raných fázích Android vývoje
- Aplikace je zde rozdělena na několik hlavních balíčků čistě podle technického typu (vrstvy)
- Všechny ViewModely žijí v jedné složce, všechna DAO rozhraní ve druhé, všechny UI obrazovky ve třetí
- Z pohledu architektury to odpovídá našemu rozdělení (UI vrstva, Datová vrstva), takže to na první pohled dává smysl
- Jakmile ale aplikace naroste na desítky obrazovek, tato iluze uspořádanosti se rychle rozpadne

Package by Layer – Ukázka struktury

- Všimněte si, jak jsou soubory patřící k jedné funkci (např. Kapybara) rozesety napříč celým projektem

```
com.example.app
|-- data/
|   |-- CapybaraDao.kt
|   |-- SettingsDao.kt
|   |-- AppDatabase.kt
|-- repository/
|   |-- CapybaraRepository.kt
|   |-- SettingsRepository.kt
|-- viewmodel/
|   |-- CapybaraViewModel.kt
|   |-- SettingsViewModel.kt
|-- ui/
|   |-- CapybaraScreen.kt
|   |-- SettingsScreen.kt
```

Hlavní nevýhody přístupu by Layer

- **Nízká soudržnost (Low Cohesion):** Soubory, které se mění společně (např. při úpravě detailu kapybary), jsou uloženy daleko od sebe

Hlavní nevýhody přístupu by Layer

- **Nízká soudržnost (Low Cohesion):** Soubory, které se mění společně (např. při úpravě detailu kapybary), jsou uloženy daleko od sebe
- **Navigační peklo:** Vývojář musí v IDE neustále otevírat a scrollovat mnoha různými složkami, když pracuje na jediné obrazovce

Hlavní nevýhody přístupu by Layer

- **Nízká soudržnost (Low Cohesion):** Soubory, které se mění společně (např. při úpravě detailu kapybary), jsou uloženy daleko od sebe
- **Navigační peklo:** Vývojář musí v IDE neustále otevírat a scrollovat mnoha různými složkami, když pracuje na jediné obrazovce
- **Rozbité zapouzdření:** Třídy v datové vrstvě (např. `CapbaraDao`) musí být často veřejné (`public`), aby k nim repozitář z jiného balíčku vůbec mohl přistoupit

Hlavní nevýhody přístupu by Layer

- **Nízká soudržnost (Low Cohesion):** Soubory, které se mění společně (např. při úpravě detailu kapybary), jsou uloženy daleko od sebe
- **Navigační peklo:** Vývojář musí v IDE neustále otevírat a scrollovat mnoha různými složkami, když pracuje na jediné obrazovce
- **Rozbité zapouzdření:** Třídy v datové vrstvě (např. `CapbaraDao`) musí být často veřejné (`public`), aby k nim repozitář z jiného balíčku vůbec mohl přistoupit
- **Blokace modularizace:** Nelze jednoduše vzít všechny soubory týkající se „Nastavení“ a přesunout je do samostatného modulu pro zrychlení kompilace, protože jsou pevně vpletené do globálních vrstev

Package by Feature (Moderní standard)

- Soubory jsou rozděleny podle toho, k jaké funkcionalitě či doméně patří (podle toho, „co soubor dělá“)
- Vytváříme balíčky pro konkrétní logické celky (např. `cap_ybara`, `settings`, `auth`)
- V rámci této složky si teprve vytvoříme miniaturní rozvrstvení na `ui` a `data`

Package by Feature (Moderní standard)

- Soubory jsou rozděleny podle toho, k jaké funkcionalitě či doméně patří (podle toho, „co soubor dělá“)
- Vytváříme balíčky pro konkrétní logické celky (např. `capybara`, `settings`, `auth`)
- V rámci této složky si teprve vytvoříme miniaturní rozvrstvení na `ui` a `data`
- Výhody tohoto přístupu:
 - ▶ **Maximální přehlednost:** Všechny soubory potřebné pro jednu obrazovku máme na jednom místě
 - ▶ **Lepší spolupráce:** Více vývojářů může pracovat na různých funkcích bez konfliktů (tzv. merge conflicts)
 - ▶ **Škálovatelnost:** Aplikace je připravena na pozdější rozdělení do více Gradle modulů

Package by Feature – Struktura projektu

- Ukázka moderní organizace kódu v Jetpack Compose aplikaci
- Složka `core` obsahuje věci sdílené napříč celou aplikací, složka `feature` obsahuje izolované funkce

```
com.example.app
|-- MyApplication.kt
|-- MainActivity.kt
|-- core/
|   |-- database/ (AppDatabase.kt)
|-- feature/
|   |-- capybara/
|   |   |-- data/
|   |   |   |-- local/ (CapybaraItem.kt, CapybaraDao.kt)
|   |   |   |-- repository/ (CapybaraRepository.kt)
|   |   |-- ui/ (CapybaraScreen.kt, CapybaraViewModel.kt)
|   |-- settings/
|   |   |-- data/
```

Od balíčků k modulům (Modularizace)

- Zpočátku naprosto stačí mít funkce oddělené jen složkami (balíčky) v jednom hlavním modulu `app`
- Jakmile ale aplikace vyrostе (desítky obrazovek, dlouhé časy kompilace), složku s funkcí jednoduše „vytrhneme“ a přesuneme do vlastního Gradle modulu

Od balíčků k modulům (Modularizace)

- Zpočátku naprosto stačí mít funkce oddělené jen složkami (balíčky) v jednom hlavním modulu `app`
- Jakmile ale aplikace vyrostе (desítky obrazovek, dlouhé časy kompilace), složku s funkcí jednoduše „vytrhneme“ a přesuneme do vlastního Gradle modulu
- V tu chvíli začne naplno fungovat modifikátor viditelnosti `internal`:

```
// Viditelne pro zbytek aplikace (verejne API modulu)
```

```
class CapybaraRepository(private val dao: CapybaraDao)
```

```
// Skryto uvnitr modulu, zbytek aplikace o DAO vubec nevi
```

```
internal interface CapybaraDao { ... }
```

```
internal data class CapybarItem( ... )
```